

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

Студентка: Белакова Полина Вячеславовна

Ст.билет: 1032252589

Группа: НКАбд-01-25

МОСКВА

2025 г

Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

Порядок выполнения лабораторной работы

Реализация подпрограмм в NASM

1. Создаю каталог для выполнения лабораторной работы № 9, перехожу в него и создаю файл lab09-1.asm (рисунок 1).

```
pvelakova2@localhost-live:~$ mkdir ~/work/arch-pc/lab09
pvelakova2@localhost-live:~$ cd ~/work/arch-pc/lab09
pvelakova2@localhost-live:~/work/arch-pc/lab09$ touch lab09-1.asm
pvelakova2@localhost-live:~/work/arch-pc/lab09$
```

Рисунок 1 - создаю каталог, перехожу в него и создаю файл.

2. Ввожу в файл lab09-1.asm текст программы из листинга 9.1 рисунок 2).

```
mc [pvelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/bin/mc
lab09-1.asm  [-M--] 27 L:[ 1+34 35/ 35
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x:',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit
;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
mov ebx, 2
mul ebx
add eax, 7
mov [res], eax
ret ; выход из подпрограммы
```

Рисунок 2 - код из листинга 9.1.

Создаю исполняемый файл и проверяю его работу рисунок 3).

```
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 1
2x+7=9
pvbelakova2@localhost-live:~/work/arch-pc/lab09$
```

Рисунок 3 - создание исполняемого файла и проверка его работы.

Изменяю текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$ (рисунок 4).

```
mc [pvbelakova2@localhost-live:~/work/arch-pc/lab09 —
lab09-1.asm      [-M--] 13 L:[ 1+35 36/ 38
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB 'f(g(x))=2*(3x-1)+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
; Основная программа
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [res]
call iprintLF
call quit
_calcul:
push eax.
call _subcalcul
mov ebx, 2 ; f(x) = 2x+7
mul ebx ; 2 * g(x)
add eax, 7 ; 2*g(x) + 7
mov [res], eax ; Сохраняем результат
pop eax
ret.
_subcalcul:
mov ebx, 3
mul ebx ; 3*x
sub eax, 1 ; 3x-1
ret.
```

Рисунок 4 - Изменяю текст программы.

Создаю исполняемый файл и проверяю его работу рисунок 5).

```

pvbelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm -o lab09-1_1.o
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1_1 lab09-1_1.o
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab09-1_1
Введите x: 1
f(g(x))=2*(3x-1)+7=11
pvbelakova2@localhost-live:~/work/arch-pc/lab09$

```

Рисунок 5 - создание исполняемого файла и проверка его работы.

Отладка программ с помощью GDB

Создаю файл lab09-2.asm с текстом программы из Листинга 9.2 (рисунок 6).

```

mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /us
lab09-2.asm [-M--] 10 L:[ 1+14
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80

```

Рисунок 6 - код из листинга 9.2.

Создаю исполняемый файл и загружаю исполняемый файл в отладчик gdb, проверяю работу программы, запустив ее в оболочке GDB с помощью команды run (рисунок 7).

```
pvmelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
pvmelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
pvmelakova2@localhost-live:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /home/pvmelakova2/work/arch-pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000
Hello, world!
[Inferior 1 (process 117906) exited normally]
(gdb) █
```

Рисунок 7 - создание исполняемого файла и запуск программы в оболочке gdb.

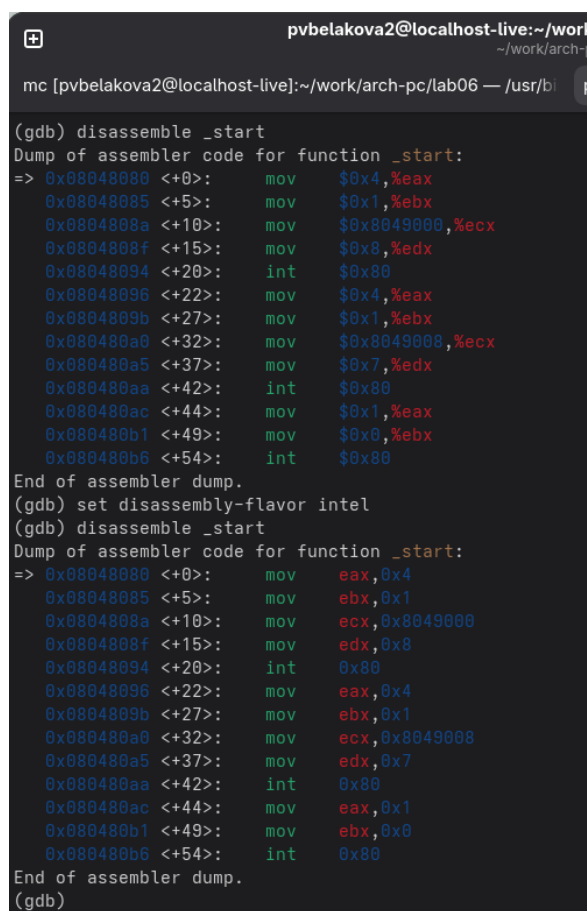
Устанавливаю брейкпоинт на метку _start и запускаю программу (рисунок 8).

```
(gdb) break _start
Breakpoint 1 at 0x8048080: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/pvmelakova2/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb) █
```

Рисунок 8 - устанавливаю брейкпоинт на метку _start.

Посматриваю дисассимилированный код программы с помощью команды `disassemble` начиная с метки `_start`, переключаюсь на отображение команд с Intel'овским синтаксисом и просматриваю дисассимилированный код (рисунок 9).



```
pvtbelakova2@localhost-live:~/work
~/work/arch-
mc [pvtbelakova2@localhost-live]:~/work/arch-pc/lab06 — /usr/bi

(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:    mov     $0x4,%eax
      0x08048085 <+5>:    mov     $0x1,%ebx
      0x0804808a <+10>:   mov     $0x8049000,%ecx
      0x0804808f <+15>:   mov     $0x8,%edx
      0x08048094 <+20>:   int     $0x80
      0x08048096 <+22>:   mov     $0x4,%eax
      0x0804809b <+27>:   mov     $0x1,%ebx
      0x080480a0 <+32>:   mov     $0x8049008,%ecx
      0x080480a5 <+37>:   mov     $0x7,%edx
      0x080480aa <+42>:   int     $0x80
      0x080480ac <+44>:   mov     $0x1,%eax
      0x080480b1 <+49>:   mov     $0x0,%ebx
      0x080480b6 <+54>:   int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:    mov     eax,0x4
      0x08048085 <+5>:    mov     ebx,0x1
      0x0804808a <+10>:   mov     ecx,0x8049000
      0x0804808f <+15>:   mov     edx,0x8
      0x08048094 <+20>:   int     0x80
      0x08048096 <+22>:   mov     eax,0x4
      0x0804809b <+27>:   mov     ebx,0x1
      0x080480a0 <+32>:   mov     ecx,0x8049008
      0x080480a5 <+37>:   mov     edx,0x7
      0x080480aa <+42>:   int     0x80
      0x080480ac <+44>:   mov     eax,0x1
      0x080480b1 <+49>:   mov     ebx,0x0
      0x080480b6 <+54>:   int     0x80
End of assembler dump.
(gdb)
```

Рисунок 9 - дисассимилированный код программы в режимах АТТ и Intel.

Различия между синтаксисом АТТ и Intel заключаются в порядке операндов (АТТ - Операнд источника указан первым. Intel - Операнд назначения указан первым).

Включаю режим псевдографики для анализа программы (рисунок 10).

```
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab06 — /usr/bi pvbelakova2@

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcf80 0xffffcf80
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>

B+>0x8048080 <_start>    mov    eax,0x4
0x8048085 <_start+5>    mov    ebx,0x1
0x804808a <_start+10>   mov    ecx,0x8049000
0x804808f <_start+15>   mov    edx,0x8
0x8048094 <_start+20>   int     0x80
0x8048096 <_start+22>   mov    eax,0x4
0x804809b <_start+27>   mov    ebx,0x1
0x80480a0 <_start+32>   mov    ecx,0x8049008
0x80480a5 <_start+37>   mov    edx,0x7
0x80480aa <_start+42>   int     0x80

native process 118163 (asm) In: _start
(gdb) layout regs
(gdb)
```

Рисунок 11 - режим псевдографики.

Добавление точек останова

С помощью команды `info breakpoints` проверяю установленные точки останова, устанавливаю еще одну точку останова по для предпоследней инструкции (`mov ebx,0x0`) (рисунок 11).


```
0x8048096 <_start+22>  mov    $0x4,%eax
0x804809b <_start+27>  mov    $0x1,%ebx
0x80480a0 <_start+32>  mov    $0x8049008,%ecx
0x80480a5 <_start+37>  mov    $0x7,%edx
0x80480aa <_start+42>  int    $0x80
0x80480ac <_start+44>  mov    $0x1,%eax
b+ 0x80480b1 <_start+49>  mov    $0x0,%ebx
0x80480b6 <_start+54>  int    $0x80

exec No process (asm) In:
(gdb)
Display all 197 possibilities? (y or n)
(gdb) i b
Num      Type          Disp Enb Address      What
1        breakpoint    keep y  0x08048080  lab09-2.asm:9
2        breakpoint    keep y  0x08048080  lab09-2.asm:9
(gdb) break *0x80480b1
Breakpoint 3 at 0x80480b1: file lab09-2.asm, line 20.
(gdb) i b
Num      Type          Disp Enb Address      What
1        breakpoint    keep y  0x08048080  lab09-2.asm:9
2        breakpoint    keep y  0x08048080  lab09-2.asm:9
3        breakpoint    keep y  0x080480b1  lab09-2.asm:20
(gdb) █
```

Рисунок 11 - просмотр установленных точек останова, установка точки по адресу инструкции.

Работа с данными программы в GDB

Изучаю содержимое регистров с помощью команды `info registers` (рисунок 12).

```
pobelakova2@localhost-live:~/work/arch-pc/lab09 — gd
~/work/arch-pc/lab09
mc [pobelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.] pobelakova2@

Register group: general
eax      0x0      0      ecx      0
edx      0x0      0      ebx      0
esp      0xffffcfa0  0xffffcfa0  ebp      0
esi      0x0      0      edi      0
eip      0x8048080  0x8048080  <_start>  eflags
cs       0x23     35      ss       0
ds       0x2b     43      es       0
fs       0x0      0      gs       0

B+>0x8048080 <_start>  mov    eax,0x4
0x8048085 <_start+5>    mov    ebx,0x1
0x804808a <_start+10>   mov    ecx,0x8049000
0x804808f <_start+15>   mov    edx,0x8
0x8048094 <_start+20>   int    0x80
0x8048096 <_start+22>   mov    eax,0x4
0x804809b <_start+27>   mov    ebx,0x1
0x80480a0 <_start+32>   mov    ecx,0x8049008
0x80480a5 <_start+37>   mov    edx,0x7
0x80480aa <_start+42>   int    0x80
0x80480ac <_start+44>   mov    eax,0x1

native process 120313 (asm) In: _start
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0  0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080  0x8048080 <_start>
eflags   0x202     [ IF ]
cs       0x23     35
ss       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--
```

Рисунок 12 - просмотр содержимого регистров.

Смотрю значение переменной msg1 по имени и значение переменной msg2 по адресу (рисунок 13).

```
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc. p

Register group: general
eax      0x0      0      ecx
edx      0x0      0      ebx
esp      0xffffcfa0 0xffffcfa0  ebp
esi      0x0      0      edi
eip      0x8048080 0x8048080 <_start>  eflags
cs       0x23     35     ss
ds       0x2b     43     es
fs       0x0      0      gs

B+> 0x8048080 <_start>    mov    eax,0x4
     0x8048085 <_start+5>  mov    ebx,0x1
     0x804808a <_start+10>  mov    ecx,0x8049000
     0x804808f <_start+15>  mov    edx,0x8
     0x8048094 <_start+20>  int     0x80
     0x8048096 <_start+22>  mov    eax,0x4
     0x804809b <_start+27>  mov    ebx,0x1
     0x80480a0 <_start+32>  mov    ecx,0x8049008
     0x80480a5 <_start+37>  mov    edx,0x7
     0x80480aa <_start+42>  int     0x80
     0x80480ac <_start+44>  mov    eax,0x1

native process 120313 (asm) In: _start
(gdb) x/1sb &msg1
0x8049000 <msg1>:      "Hello, "
(gdb) x/1sb 0x8049008
0x8049008 <msg2>:      "world!\n\034"
(gdb)
```

Рисунок 13 - просмотр содержимого двух переменных.

Изменяю первый символ переменной msg1 и второй символ во второй переменной msg2 (рисунок 14).

```
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/bin

Register group: general
eax      0x0      0
edx      0x0      0
esp      0xffffcfa0 0xffffc
esi      0x0      0
eip      0x8048080 0x80480
cs       0x23     35
ds       0x2b     43
fs       0x0      0

0x80480b6 <_start+54>   int    0x80
0x80480b8               add     BYTE PTR
0x80480ba               add     BYTE PTR
0x80480bc               add     BYTE PTR
0x80480be               add     BYTE PTR
0x80480c0               add     BYTE PTR
0x80480c2               add     BYTE PTR
0x80480c4               add     BYTE PTR
0x80480c6               add     BYTE PTR
0x80480c8               add     BYTE PTR
0x80480ca               add     BYTE PTR

native process 120313 (asm) In: _start
(gdb) x/1sb &msg1
0x8049000 <msg1>:      "Hello, "
(gdb) x/1sb &msg2
0x8049008 <msg2>:      "world!\n\034"
(gdb) set {char}&msg1='h'
(gdb) set {char}0x8049009='0'
(gdb) x/1sb &msg1
0x8049000 <msg1>:      "hello, "
(gdb) x/1sb &msg2
0x8049008 <msg2>:      "w0rld!\n\034"
(gdb) █
```

Рисунок 14 - замена символов.

С помощью команды set изменяю значение регистра ebx (рисунок 15).

```
mc [pvbelakova2@localhost-live:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.] pvbelakova2@localhost-live:~/work/arch-pc/lab09 — gdb lab09

Register group: general
eax      0x0      0      ecx      0x0      0
edx      0x0      0      ebx      0x2      2
esp      0xffffcfa0 0xffffcfa0  ebp      0x0      0x0
esi      0x0      0      edi      0x0      0
eip      0x8048080 0x8048080 <_start>  eflags    0x202     [ IF ]
cs       0x23     35     ss       0x2b     43
ds       0x2b     43     es       0x2b     43
fs       0x0      0      gs       0x0      0

0x80480b6 <_start+54>  int    0x80
0x80480b8             add    BYTE PTR [eax],al
0x80480ba             add    BYTE PTR [eax],al
0x80480bc             add    BYTE PTR [eax],al
0x80480be             add    BYTE PTR [eax],al
0x80480c0             add    BYTE PTR [eax],al
0x80480c2             add    BYTE PTR [eax],al
0x80480c4             add    BYTE PTR [eax],al
0x80480c6             add    BYTE PTR [eax],al
0x80480c8             add    BYTE PTR [eax],al
0x80480ca             add    BYTE PTR [eax],al

native process 120313 (asm) In: _start L9 PC: 0
(gdb) set $ebx='2'
(gdb) p/s $ebx
$2 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$3 = 2
(gdb)
```

Рисунок 15 - изменение значение регистра ebx.

Команда p/s всегда выводит значение в десятичном формате как знаковое целое, независимо от того, как оно было установлено (50 — это ASCII-код символа '2', 2 — это просто число 2).

Завершаю выполнение программы с помощью команды continue (сокращенно c) и выхожу из GDB с помощью команды quit (сокращенно q) (рисунок 16).

```
(gdb) chello, wOrld!
Continuing.

Breakpoint 3, _start () at lab09-2.asm:20
(gdb) q
A debugging session is active.

        Inferior 1 [process 120313] will be killed.

Quit anyway? (y or n) █
```

Рисунок 16 - завершение и выход из gdb.

Обработка аргументов командной строки в GDB

Копирую файл lab8-2.asm, созданный при выполнении лабораторной работы №8 в файл с именем lab09-3.asm (рисунок 17).

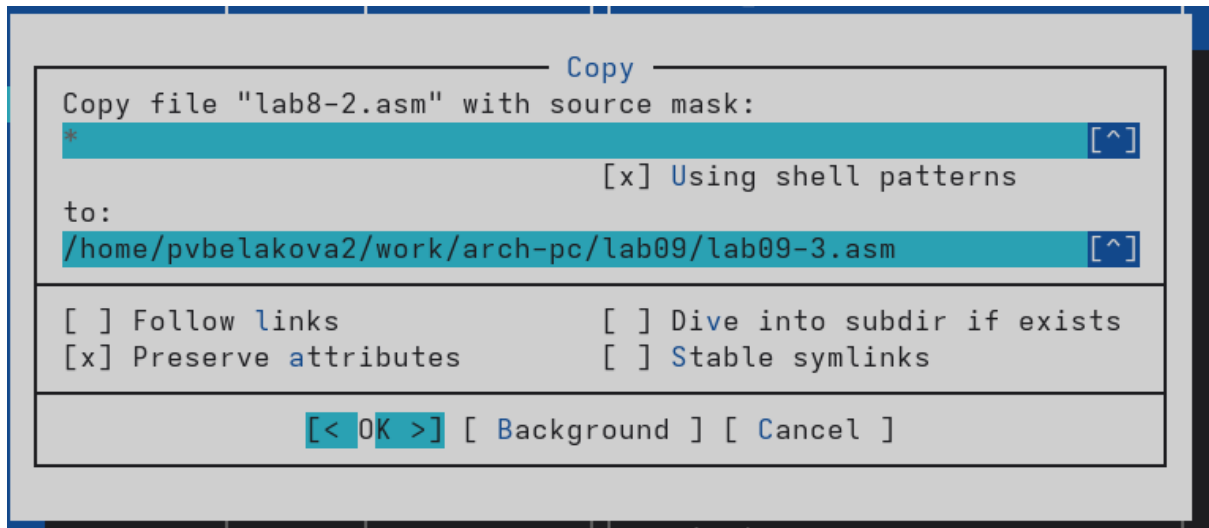


Рисунок 17 - копирование файла.

Создаю исполняемый файл, загружаю исполняемый файл в отладчик, указав аргументы (рисунок 18).

```
pvelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
pvelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
pvelakova2@localhost-live:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb)
```

Рисунок 18 - создание исполняемого файла и его загрузка в отладчик.

Устанавливаю точку останова перед первой инструкцией в программе и запускаю ее. Как видно, число аргументов равно 5 – это имя программы lab09-3 и непосредственно аргументы: аргумент1, аргумент, 2 и 'аргумент 3'. Просматриваю остальные позиции стека (рисунок 19).

```
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab09-3.asm, line 8.
(gdb) run
Starting program: /home/pvbelakova2/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7fc000
Breakpoint 1, _start () at lab09-3.asm:8
8      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp~
A syntax error in expression, near `~'.
(gdb) x/x $esp
0xffffcf60: 0x00000005
(gdb) x/s *(void**)(esp + 4)
0xffffd137: "/home/pvbelakova2/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xffffd164: "аргумент1"
(gdb) x/s *(void**)(esp + 12)
0xffffd176: "аргумент"
(gdb) x/s *(void**)(esp + 16)
0xffffd187: "2"
(gdb) x/s *(void**)(esp + 20)
0xffffd189: "аргумент 3"
(gdb) x/s *(void**)(esp + 24)
0x0: <error: Cannot access memory at address 0x0>
(gdb) █
```

Рисунок 19 - установка точки останова, запуск программы, просмотр позиций стека.

Шаг изменения адреса равен 4 байтам, потому что на архитектуре x86 (32-битной) размер указателя (адреса) составляет 4 байта.

Задание для самостоятельной работы

1. Преобразовываю программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму (рисунок 20).

```
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /u
lab8-task.asm [-M--] 13 L:[ 1+30
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start

f_calc:
    add eax, 2      ; x + 2
    imul eax, 5     ; (x + 2) * 5
    ret

; --- Основная программа ---
_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0
next:
    cmp ecx, 0h
    jz _end
    pop eax
    call atoi
    call f_calc
    add esi, eax
    loop next
_end:
    mov eax, msg
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

Рисунок 20 - преобразованный код программы из лабораторной работы 8 с использованием подпрограммы.

Создаю исполняемый файл и проверяю его работу (рисунок 21).


```

pvbelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf lab8-task.asm
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab8-task lab8-task.o
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab8-task
Результат: 0
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab8-task 4
Результат: 30
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab8-task 1
Результат: 15
pvbelakova2@localhost-live:~/work/arch-pc/lab09$

```

Рисунок 21 - создание исполняемого файла и проверка его работы.

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Создаю файл lab09-task_2.asm и ввожу в него код из листинга 9.3 (рисунок 22).

```

mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/
lab09-task_2.asm [-M--] 9 L:[ 1+19 2
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit

```

Рисунок 22 - код из листинга 9.3.

Создаю исполняемый файл и загружаю его в отладчик gdb (рисунок 23).

```
pvelakova2@localhost-live:~/work/arch-pc/lab09$ touch lab09-task_2.asm
pvelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-task_2.lst lab09-task_2.asm
pvelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-task_2 lab09-task_2.o
pvelakova2@localhost-live:~/work/arch-pc/lab09$ gdb lab09-task_2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-task_2...
(gdb) █
```

Рисунок 23 - создание исполняемого файла и запуск программы в оболочке gdb.

Проверяю работу программы пошагово (рисунок 24).

```
pobelakova2@localhost-live:~/work/arch-pc/lab09 — gdb lab09-task_2
~/work/arch-pc/lab09
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /usr/bin/mc -P /tmp/mc.1
pvbelakova2@localhost-live:~/work/arch-pc/lab09 — gdb lab09-task_2

eax      group: general      8      ecx      0x4      4
eax      0x8      8      ecx      0x4      4
edx      0x0      0      ebx      0xa      10
esp      0xffffcfa0      0xffffcfa0      ebp      0x0      0x0
esi      0x0      0      edi      0x0      0
eip      0x804817e      0x804817e <_start+ eflags 0x10206 [ PF IF RF ]
cs      0x23      35      ss      0x2b      43
ds      0x2b      43      es      0x2b      43
fs      0x0      0      gs      0x0      0

B+ 0x8048168 <_start>      mov     ebx,0x3
0x804816d <_start+5>      mov     eax,0x2
0x8048172 <_start+10>     add     ebx,eax
0x8048174 <_start+12>     mov     ecx,0x4
0x8048179 <_start+17>     mul     ecx
0x804817b <_start+19>     add     ebx,0x5
>0x804817e <_start+22>     mov     edi,ebx
0x8048180 <_start+24>     mov     eax,0x8049000 >
0x8048185 <_start+29>     call    0x804808f <sprint>
0x804818a <_start+34>     mov     eax,edi      LF>
0x804818c <_start+36>     call    0x8048106 <iprintf>

native process 132135 (asm) In: _start      L14      PC: 0x804817e
(gdb) rprocess 132237 (asm) In: _start      L14      PC: 0x804817e
Continuing.
[Inferior 1 (process 132135) exited normally]
(gdb) run
Starting program: /home/pvbelakova2/work/arch-pc/lab09/lab09-task_2

Breakpoint 1, _start () at lab09-task_2.asm:8
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
```

Рисунок 24 - Проверка работы программы по шагам.

На основе отладки изменяю код программы (рисунок 25).

```
mc [pvbelakova2@localhost-live]:~/work/arch-pc/lab09 — /u
lab09-task_2.asm [----] 9 L:[ 1+19
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov eax,3
mov ebx,2
add eax,ebx
mov ebx,4
mul ebx
add eax,5
mov edi,eax
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 25 - измененный код программы из листинга 9.3.

Создаю исполняемый файл и проверяю его работу (рисунок 26).

```
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-task_2.lst lab09-task_2.asm
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-task_2 lab09-task_2.o
pvbelakova2@localhost-live:~/work/arch-pc/lab09$ ./lab09-task_2
Результат: 25
pvbelakova2@localhost-live:~/work/arch-pc/lab09$
```

Рисунок 26 - создание исполняемого файла и проверка его работы.

Выводы

В ходе лабораторной работы были приобретены навыки написания программ с использованием подпрограмм, ознакомились с методами отладки при помощи GDB и его основными возможностями.